Implementation of a CAN/CAN-FD Bus Transmitter in VHDL

Mads Richardt (s224948)

February 28, 2026

Contents

1 Design and Architecture **1**
1.1 System Overview 1
1.1.1 Interface Definition Tables 1
1.2 LLC Sub-layer (`tx_llc`) 8
1.3 MAC Sub-layer (`tx_mac`) 8
1.4 PCS Sub-layer (`tx_pcs`) 8

1 Design and Architecture

1.1 System Overview

A complete CAN node is composed of a Transmit (TX) path and a Receive (RX) path coordinated by shared fault-confinement and physical-interface control. The paths operate in parallel at runtime, but they are not symmetric in behavior or responsibility.

The focus of this thesis is the complete node architecture and its standards-traceable interfaces. The TX path is responsible for frame submission, arbitration participation, and bitstream generation toward the bus, while the RX path is responsible for bus observation, frame reconstruction, and delivery/notification toward the user layer. As shown in Figure 1, this full-node decomposition spans LLC, MAC, PCS, FCE, and PMA boundaries.

1.1.1 Interface Definition Tables

The following tables define the interface bundles shown in Figure 1. ISO references are to ISO 11898-1:2024 clauses and tables.

Detailed interface definitions (normative source mapping):

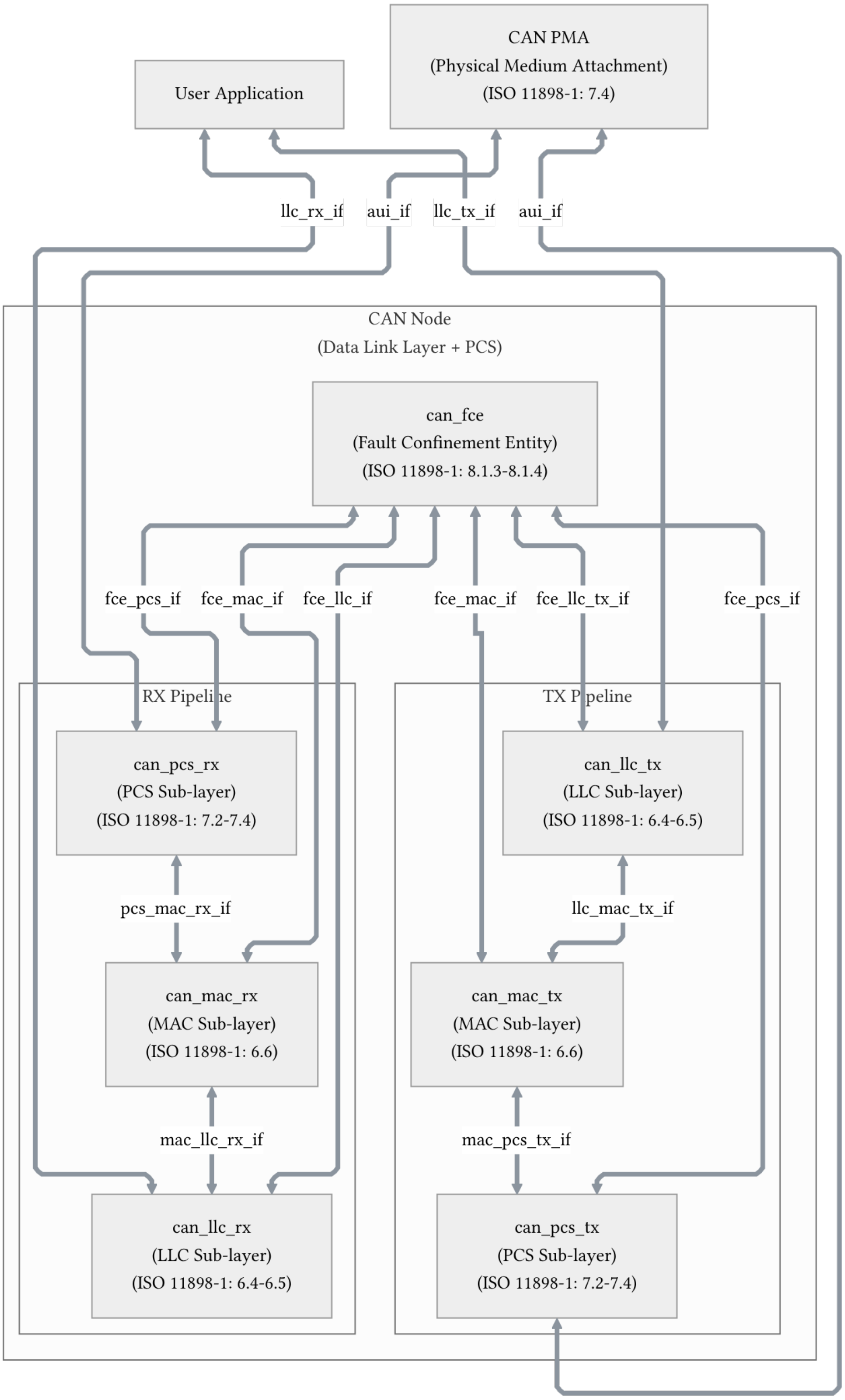

Figure 1: CAN node decomposition across LLC, MAC, PCS, FCE, and PMA boundaries.

1.1.1.1 `llc_tx_if` (User -> `can_llc_tx`)

Table 1: Interface definition for `llc_tx_if` (User to `can_llc_tx`).

Signal/primitive	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`L_Data.Request` (start)	User -> LLC	Request	Request data transfer of one LLC frame (`Frame`, `Handle`)	Start of transfer transaction	6.4.5.5, Table 6, Table 7, L_Data.Request semantics	`valid && ready && sop` on first Avalon-ST beat
`L_Data.Request` (payload)	User -> LLC	Stream data	Continuation of frame transfer	Intermediate beats while frame is streamed	6.4.3, 6.4.5.5	`valid && ready && !sop && !eop` with payload on `data`
`L_Data.Request` (end)	User -> LLC	End marker	End of current frame transfer	Final beat of transfer transaction	6.4.5.5 (implementation transport mapping)	`valid && ready && eop`
`L_Data.AbortRequest`	User -> LLC	Request	Abort pending frame transfer (`Handle`)	Issued by LLC user to cancel queued/re-arbitrating transfer	6.4.5.5, Table 6, Table 7, L_Data.AbortRequest semantics	Dedicated abort control signal/register (outside Avalon-ST framing)

1.1.1.2 `llc_rx_if` (`can_llc_rx` -> User)

Table 2: Interface definition for `llc_rx_if` (`can_llc_rx` to User).

Signal/primitive	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`L_Data.Indication`	LLC -> User	Indication	Received LLC frame and associated metadata (`Frame`, `Timestamp`)	Generated after MAC reception and LLC processing	6.4.5.5, Table 6, Table 7, L_Data.Indication semantics	`tbd`
`L_Data.Confirm`	LLC -> User	Confirm	Local result of previous TX request (`Transfer_Status`, `Timestamp`, `Handle`), where `Transfer_Status` is one of: ongoing, lost arbitration, transmitted, aborted, disturbed	Generated on completion/failure of local TX transaction	6.4.5.5.4, Table 7, L_Data.Confirm semantics	Confirm sideband fields: `transfer_status`, `timestamp`, `handle`

1.1.1.3 `llc_mac_tx_if` (`can_llc_tx` -> `can_mac_tx`)

Table 3: Interface definition for `llc_mac_tx_if` (`can_llc_tx` to `can_mac_tx`).

Signal/object	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`LLC frame` (DLL SDU)	LLC -> MAC	Data object	SDU carrying format/ID/control/data for MAC encapsulation	Passed when LLC initiates transfer to MAC	6.3, 6.4.3, 6.6.4.2, 6.6.9	`tbd`
Interface control information	LLC -> MAC	Control	Control context accompanying LLC frame transfer	Valid with SDU handoff	6.6.4.2	`tbd`

1.1.1.4 `mac_llc_rx_if` (`can_mac_rx` -> `can_llc_rx`)

Table 4: Interface definition for `mac_llc_rx_if` (`can_mac_rx` to `can_llc_rx`).

Signal/object	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
Reconstructed LLC frame	MAC -> LLC	Data object	MAC decapsulation result (received DF/RF mapped to LLC frame)	Generated on valid MAC frame reception	6.6.4.3, 6.6.9	`tbd`
Time reference notification	MAC -> LLC	Notification	SOF and frame-valid reference points for timestamp capture	Generated for transmitted/received DF/RF	6.6.3	`tbd`

1.1.1.5 `mac_pcs_tx_if` (`can_mac_tx` -> `can_pcs_tx`)

Table 5: Interface definition for `mac_pcs_tx_if` (`can_mac_tx` to `can_pcs_tx`).

Signal/primitive	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`PCS_Data.Request(Output_Unit)`	MAC -> PCS	Request	Request transmission of one dominant/recessive bit	Bit-by-bit during MAC serialization	7.2.1, 7.2.2	`tbd`
`PCS_Mode.Request(XL_Mode)`	MAC -> PCS	Request	Select XL mode active/passive for transceiver mode switching control	Asserted for XL frame window as defined by standard	7.2.1, 7.2.4	`tbd`
`PCS_Status.Transmitter(D_Transmit)`	MAC -> PCS	Status	Indicate data TX-mode interval for FD/XL data phase handling	Active during FD/XL data TX phase	7.2.1, 7.2.5	`tbd`

Signal/primitive	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`PCS_Status.Receiver(D_Receive)`	MAC -> PCS	Status	Indicate data RX-mode interval for FD/XL data phase handling	Active during FD/XL data RX phase	7.2.1, 7.2.6	`tbd`

1.1.1.6 `pcs_mac_rx_if` (`can_pcs_rx` -> `can_mac_rx`)

Table 6: Interface definition for `pcs_mac_rx_if` (`can_pcs_rx` to `can_mac_rx`).

Signal/primitive	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`PCS_Data.Indicate(Input_Unit)`	PCS -> MAC	Indication	Indicate arrival of one dominant/recessive bit from PL	Bit-by-bit reception stream into MAC RX path	7.2.1, 7.2.3	`tbd`

1.1.1.7 `aui_if` (PCS <-> PMA)

Table 7: AUI signal mapping between PCS and PMA.

Signal/symbol	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`output symbol`	PCS -> PMA	Symbol	Dominant/recessive physical output symbol	On `Output_Unit` from MAC via PCS	7.4.2.1	`tbd`
`mode symbol`	PCS -> PMA	Symbol	Control CAN transceiver mode switching enable/disable	On `XL_Mode` updates	7.4.2.4	`tbd`
`D_Transmit symbol`	PCS -> PMA	Symbol	Data TX-mode control symbol	On `D_Transmit` updates	7.4.2.5	`tbd`
`D_Receive symbol`	PCS -> PMA	Symbol	Data RX-mode control symbol	On `D_Receive` updates	7.4.2.6	`tbd`
`bus_off symbol`	PCS -> PMA	Symbol	Switch node off bus	On supervisor bus-off request	7.4.2.2, 8.1.3.4	`tbd`
`bus_off_release symbol`	PCS -> PMA	Symbol	Release node from bus-off	On supervisor bus-off-release request	7.4.2.3, 8.1.3.4	`tbd`
`input symbol`	PMA -> PCS	Symbol	Physical input symbol (dominant/recessive)	Continuous PMA->PCS indication	7.4.3	`tbd`

1.1.1.8 `fce_llc_tx_if` and `fce_llc_if` (LLC <-> FCE)

Table 8: FCE and LLC interface services for normal-mode and bus-off state reporting.

Signal/service	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`Normal_mode_request`	LLC -> FCE	Request	Reset FCE to initial state (error counters reset)	Startup/restart request path	8.1.3.2, Table 14	`tbd`
`Normal_mode_response`	FCE -> LLC	Response	Acknowledge normal mode request	Returned by FCE after processing	8.1.3.2, Table 15	`tbd`
`Bus_off`	FCE -> LLC	Status	Indicate node is in bus-off state	Asserted when bus-off transition occurs	8.1.3.2, Table 15; 8.1.4.4	`tbd`

1.1.1.9 `fce_mac_if` (MAC <-> FCE)

Table 9: FCE and MAC interface services for error signaling and state transitions.

Signal/service	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`Transmit/receive`	MAC -> FCE	Status	Current transfer mode context	Updated with MAC transfer context	8.1.3.3, Table 16	`tbd`
`Error`	MAC -> FCE	Event	Error detected by MAC	On bit/stuff/CRC/form/ACK error detection	8.1.3.3, Table 16	`tbd`
`Primary_error`	MAC -> FCE	Event	Primary error indication (dominant after own error flag)	On primary error condition	8.1.3.3, Table 16	`tbd`
`Error/overload flag`	MAC -> FCE	Event	MAC currently sending EF/OF	During EF/OF transmission	8.1.3.3, Table 16	`tbd`
`Counters_unchanged`	MAC -> FCE	Event/qualifier	Qualifier for counter-exception cases	On rule-c exception cases	8.1.3.3, Table 16; 8.1.4.2 rule c	`tbd`
`Error_delimiter_too_late`	MAC -> FCE	Event	Late delimiter condition after dominant-bit sequences	Set on specified late delimiter condition	8.1.3.3, Table 16	`tbd`
`Successful_transfer`	MAC -> FCE	Event	Successful TX/RX completion	On successful frame transfer outcome	8.1.3.3, Table 16	`tbd`

Signal/service	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`Error_passive_response`	MAC -> FCE	Response	MAC entered error-passive state	On state transition completion	8.1.3.3, Table 16; 8.1.4.3	`tbd`
`Error_active_response`	MAC -> FCE	Response	MAC returned to error-active state	On state transition completion	8.1.3.3, Table 16; 8.1.4.3	`tbd`
`Error_passive_request`	FCE -> MAC	Request	Request MAC enter error-passive state	On TEC/REC threshold crossing	8.1.3.3, Table 17; 8.1.4.3	`tbd`
`Error_active_request`	FCE -> MAC	Request	Request MAC return to error-active state	On TEC/REC recovery conditions	8.1.3.3, Table 17; 8.1.4.3	`tbd`

1.1.1.10 `fce_pcs_if` (FCE <-> PCS/PL)

Table 10: FCE and PCS/PL interface services for bus-off entry and recovery.

Signal/service	Direction	Type	Payload/semantics	Timing/trigger	ISO reference	Implementation mapping
`Bus_off_request`	FCE -> PL/PCS	Request	Request node switch-off from bus	On bus-off transition condition	8.1.3.4, Table 18; 8.1.4.4	`tbd`
`Bus_off_release_request`	FCE -> PL/PCS	Request	Request re-enable normal TX/RX node mode	On restart/normal-mode reintegration path	8.1.3.4, Table 18; 8.1.4.4	`tbd`
`Bus_off_response`	PL/PCS -> FCE	Response	Response to bus-off request	Returned by PL/PCS after bus-off action	8.1.3.4, Table 19	`tbd`
`Bus_off_release_response`	PL/PCS -> FCE	Response	Response to bus-off-release request	Returned by PL/PCS after release action	8.1.3.4, Table 19	`tbd`

Note on `Timestamp` and `Handle` (implementation scope):

- The ISO LLC service definitions include `Timestamp` and `Handle` parameters (see Table 7 and `L_Data.Confirm` semantics) and they are retained in these tables for standards traceability and future scalability.
- In the current implementation scope, a single TX frame buffer is used, so a multi-buffer selector `Handle` is not required.
- Timestamp capture is currently optional/out of scope; if omitted, the implementation may treat timestamp values as constant/undefined according to the selected feature subset.

1.2 LLC Sub-layer (`tx_llc`)

Responsible for frame buffering and retransmission management. It provides an Avalon-ST interface to the user application and communicates with the FCE to handle retransmission limits and error status reporting.

1.3 MAC Sub-layer (`tx_mac`)

The core of the protocol logic. It handles bit serialization, CRC calculation, and bit stuffing. It coordinates the overall frame state machine and interacts closely with the Fault Confinement Entity (FCE) to manage error counters (TEC/REC) and node state (Error Active/Passive/Bus Off) based on protocol violations.

1.4 PCS Sub-layer (`tx_pcs`)

Handles bit timing and synchronization. It generates the sample point (SP) and secondary sample point (SSP) strobes. It provides bit-level monitoring data to the FCE to detect synchronization and timing errors.
